Project Title:
WhatsOnTheMenu

Team 10

Hien Ly : Hien.ly@sjsu.edu
Sparsh Kumar : sparsh.kumar@sjsu.edu
Jayden Manning : Jayden.manning@sjsu.edu

Project Description

WhatsOnTheMenu is a smart kitchen-management web platform that helps restaurant chefs and home cooks track ingredient inventory, see which recipes they can make right now, and request restocks when running low. Chefs manage a restaurant's inventory and source restocks from Restockers, commercial suppliers who accept or reject requests. Consumers manage a personal pantry and restock through a simulated online grocery store (OGS) checkout.

Today, chefs and home cooks rely on manual tracking, memory, or checking shelves to decide what they can cook and what to reorder, leading to wasted trips, last-minute substitutions, and inefficient purchasing. WhatsOnTheMenu closes this gap by cross-referencing a user's live inventory against recipe requirements to give a real-time feasibility status for every recipe.

The primary goal is to reduce wasted ingredients, time, and purchasing by connecting inventory directly to recipe planning and restocking. The system tracks ingredient quantities, rates each recipe as makeable, almost makeable, or not makeable, and routes restocks to the right channel: a commercial Restocker for Chefs or a simulated OGS order for Consumers.

Stakeholders include Chefs (manage restaurant inventory and need reliable restock fulfillment), Consumers (home cooks managing a personal pantry), Restockers (suppliers who fulfill or reject requests and maintain their own catalog and pricing), and Administrators (oversee the global recipe/ingredient catalog, user accounts, and Restocker approval).

Our innovation is unifying personal pantry management and commercial kitchen inventory under one data model, sharing a single ingredient list and recipe structure, while distinguishing commercial restocking (accept/reject fulfillment) from personal restocking (a simulated OGS order).

System Environment

System Structure

WhatsOnTheMenu uses a three-tier architecture:

[Architecture diagram]

Client (Browser) Layer: All users access the system through a web browser. The interface is built with HTML, CSS, and JavaScript.

Application (Web) Layer: Java and JSP on Apache Tomcat handle login, sessions, role-based permissions, feasibility calculations, and restock routing.

Database Layer: MySQL stores all persistent data: users and their role subtypes (Admin, Chef, Restocker, Consumer), ingredients and restaurant stock levels, recipes and their ingredients, consumer pantries, Restocker catalogs, restock requests and their items, OGS orders and their items, and audit logs.

Hardware and Software Used

- OS: Windows / macOS / Linux
- Web Server: Apache Tomcat
- Database: MySQL Community Server
- Languages: Java, JSP, SQL, JavaScript, HTML, CSS

Functional Requirements

WhatsOnTheMenu offers different functions by role. Every user has one account and belongs to exactly one role: Chef, Consumer, Restocker, or Admin. Chefs check recipes against the shared restaurant inventory and request restocks from approved Restockers. Consumers check recipes against their personal pantry and place simulated OGS orders. Both can create private recipes, copy public ones, and add new ingredients to the shared list. Restockers maintain a priced catalog of the ingredients they supply and accept or reject incoming requests. Administrators manage the global catalog and user accounts, approve Restockers, and review audit logs.

Auditing is available to Restockers and Administrators. Restockers see requests and analytics for their own catalog only. Administrators see system-wide logs of restock decisions, inventory updates, catalog edits, and account changes, each recording who acted, on what record, what was done, and when.

Chef View

User Authentication

- Chefs log in with an email and password before submitting requests or updating inventory, keeping data accountable.
- Passwords are hashed, and login creates a secure session. Suspended accounts cannot log in.
- Logged-out visitors can browse the public recipe library but cannot view inventory or submit requests.

Inventory Management

- Chefs view and update the restaurant's inventory, tracked as a quantity on hand per ingredient in its default unit.
- Each update records a timestamp and an audit log entry, so consumption can be tracked over time.

Recipe Library

- Chefs can browse public recipes and save a private copy, including all ingredients and quantities, with themselves as creator.

- Chefs can create private recipes from scratch with a name, description, and each ingredient's quantity and unit.
- When adding ingredients, Chefs can pick an existing one or add a new one to the shared list for future use. Ingredient names are unique to prevent duplicates.
- Chefs can edit their own recipes at any time, adding, removing, or adjusting ingredients.

Recipe Feasibility Check

- The system compares each recipe's required quantities against the restaurant's stock (recorded in matching units) and shows:
    - Makeable: all ingredients in stock
    - Almost Makeable: 1–2 ingredients short
    - Not Makeable: 3 or more ingredients short or missing
- Feasibility is recalculated in real time.

Simple Restock Request

- A Chef can request a specific ingredient and quantity from an approved Restocker of their choice, without a recipe.
- The system confirms the Restocker carries that ingredient before submitting.

Recipe-Based Restock Request

- A Chef selects a recipe, and the system computes the shortfall against current inventory.
- Any missing quantities are bundled into one request, linked to the recipe, and sent to the Chef's chosen Restocker. If nothing is short, the Chef is told no restock is needed.

Request Status Tracking

- Every request moves through Pending > Accepted or Rejected > Fulfilled, with submission and response times recorded.
- Chefs can view the status and history of all their requests.

Consumer View

User Authentication

- Consumers log in like Chefs, with hashed passwords and session-based authentication.

Pantry Management

- Consumers manage a personal pantry, separate from the restaurant inventory, recording each ingredient's quantity, unit, and last update.

Recipe Library

- Consumers can copy public recipes, create private ones from scratch, and add new ingredients on the fly, just like Chefs.

Recipe Feasibility Check

- Consumers get the same Makeable / Almost Makeable / Not Makeable check, calculated against their pantry.

Simulated OGS Order

- Consumers missing ingredients can place a simulated OGS order. No real grocery service is contacted.
- Each order records its status, time placed, and the ingredients and quantities ordered. On simulated fulfillment, those quantities are added to the pantry automatically.

Restocker View

Restocker Login

- Restocker accounts include a company name and are linked to the ingredients they supply. They must be approved by an Administrator before receiving requests.

Catalog and Pricing Management

- Restockers add, edit, and remove items in their own catalog, including price and available quantity. Removing an item doesn't delete the ingredient from the system.
- Restockers cannot view or edit other Restockers' catalogs.

Review, Accept, and Reject Requests

- Restockers see all pending requests directed to them and can accept or reject each one. The decision updates the status, records the response time, is logged, and appears in the Chef's request history.
- When an accepted request is marked Fulfilled, the quantities are added to the restaurant's inventory and the transaction is logged.

View Operational Analytics

- Dashboards show request volume over time, most-requested ingredients, and accept/reject rates to support inventory and pricing planning.

Administrator View

Manage Global Recipe/Ingredient Catalog

- Administrators add, edit, and remove public recipes and shared ingredients.

Manage User Accounts

- Administrators can suspend abusive accounts and review activity through the audit log.

Approve / Vet Restockers

- New Restockers need Administrator approval before receiving requests. The system records who approved them and when.

Audit and Accountability

- Administrators can view system-wide audit logs of restock decisions, account changes, inventory updates, and catalog edits, ensuring accountability across all roles.

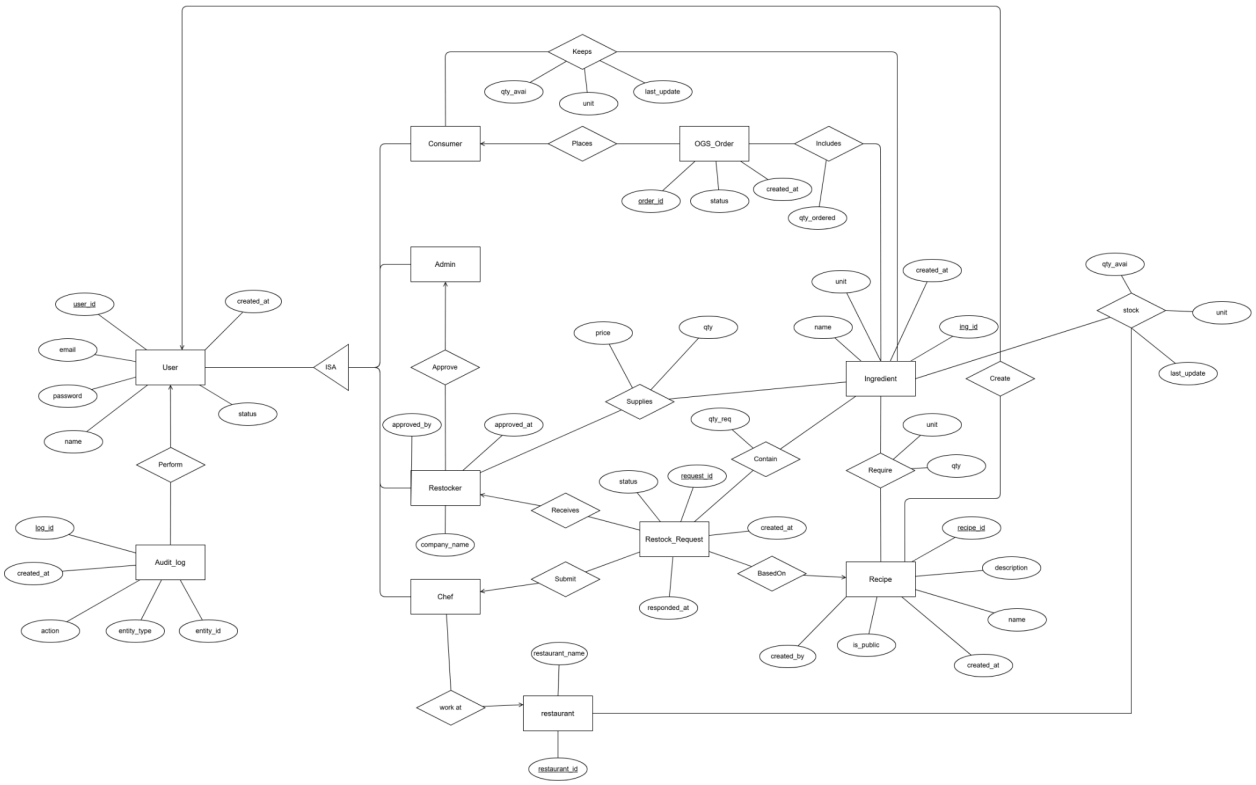
Non-Functional Issues

Access Control

- Role-based access control limits each user to actions tied to their role.
- Chefs manage the restaurant inventory and their own recipes and requests. Consumers manage only their own pantry, recipes, and orders.
- Restockers manage only their own catalog and the requests sent to them.
- Administrators have full access to the global catalog, user accounts, and Restocker approvals.

Data Integrity

- Each user belongs to exactly one role, and each role record maps to exactly one account.
- Unique ingredient names prevent duplicates when adding ingredients while building a recipe.
- An ingredient can't appear twice in the same recipe, pantry, catalog, restock request, or OGS order.
- A request can be accepted or rejected only once, and only accepted requests can be fulfilled, preventing duplicate or conflicting fulfillment.
- These constraints keep feasibility calculations and request history accurate.



Entities, Attributes, Relationships

users

- User is a superclass of admin, chef, restocker, and consumer. Every user must belong to exactly one of these four subtypes. Each Admin, Chef, Restocker, or Consumer must correspond to exactly one User. The subtype entities use user_id as both their PK and FK referencing users(user_id).

admin

- user_id is the PK of this entity, and is also a FK referencing users(user_id), which is how Admin inherits all its base fields from User. This entity has no attributes of its own since an Admin's role doesn't require any extra data beyond what User already stores. This entity is a child of the ISA relationship from User, and is on the "one" side of Approve (an admin approves many restockers).

chef

- user_id is the PK of this entity, and is also a FK referencing users(user_id), inheriting the base fields from User. This entity currently has no attributes of its own. This entity is a child of the ISA relationship from User, and is on the "one" side of Submit (a chef submits many restock requests). It also contain an FK from the relationship with Restaurant which is restaurant_id.

restaurant

- restaurant_id is the PK of this entity. This entity also has a name attribute but it is not a PK because many restaurants can have the same name. This entity has a many to one relationship with Chef, and is on the "one" side of Restaurant (a restaurant can have many chefs but a chef can only be in 1 restaurant).

restocker

- user_id is the PK of this entity, and is also a FK referencing users(user_id), inheriting the base fields from User. company_name identifies the business so Chefs know who they're requesting from. This entity is a child of the ISA relationship from User, and is on the "many" side of Approve. It's also on the "one" side of Receives (a restocker receives many restock requests) and on the "many" side of Supplies (a restocker supplies many ingredients).

consumer

- user_id is the PK of this entity, and is also a FK referencing users(user_id), inheriting the base fields from User. This entity currently has no attributes of its own. This entity is a child of the ISA relationship from User, and is on the "one" side of Places (a consumer places many OGS orders) and on the "many" side of Keeps (a consumer keeps many ingredients in their pantry).

ingredient

- ing_id is the PK of this entity. The user can choose an appropriate unit for the ingredient through default_unit when they add one. created_at will be used for audit_log. The name attribute is for the user to know what the ingredient is, and is kept unique so the same ingredient can't be entered twice. This entity is on the "many" side of Require (with Recipe), Contain (with Restock_Request), Includes (with OGS_Order), Supplies (with Restocker), Stock (with Restaurant), and Keeps (with Consumer).

recipe

- recipe_id is the PK of this entity. name and description let the user identify what the recipe makes and how. is_public marks whether the recipe belongs to the shared/global library or stays private to its creator. created_by is a FK referencing users(user_id), recording who made the recipe. created_at is used for audit purposes. This entity is on the "many" side of Create (with User), on the "many" side of Require (with Ingredient), and on the "one" side of BasedOn (a recipe can underlie many restock requests).

restock_request

- request_id is the PK of this entity. status tracks the request through its lifecycle: PENDING, ACCEPTED, REJECTED, or FULFILLED. chef_id is a FK referencing chef(user_id), recording who submitted the request. restocker_id is a FK referencing restocker(user_id), recording who the request was sent to. recipe_id is a FK referencing recipe(recipe_id), and is nullable; it's set when the request came from a recipe-based shortfall and left null for a simple request. created_at and responded_at are used for audit purposes.

audit_log

- log_id is the PK of this entity. user_id is a FK referencing users(user_id), recording who performed the action. entity_type and entity_id identify which record the action was performed on. action describes what was done. created_at records when it happened.

Create

- Create is a relationship between User and Recipe. This is a one-to-many relationship because one user can create many recipes, but each recipe has exactly one creator. Its FK is recipe.created_by, referencing users(user_id)

Perform

- Perform is a relationship between User and Audit_log. This is a one-to-many relationship because one user can perform many logged actions, but each audit log entry has exactly one acting user. Its FK is audit_log.user_id, referencing users(user_id)..

Work at

- Work is a relationship between chef and restaurant. This is a one-to-many relationship because one restaurant can have many chefs, but each chef works for exactly one restaurant. Its FK is restaurant_id, referencing restaurant (restaurant _id). This relationship does not require a table and is included in the chef table.

Approve

- Approve is a relationship between Admin and Restocker. This is a one-to-many relationship because one admin can approve many restockers, but each restocker has zero or one approving admin. Its FK is restocker.approved_by, referencing admin(user_id); it's nullable because a restocker may not be approved yet.

Submit

- Submit is a relationship between Chef and Restock_Request. This is a one-to-many relationship because one chef can submit many restock requests, but each request has exactly one submitting chef. Its FK is restock_request.chef_id, referencing chef(user_id).

Receives

- Receives is a relationship between Restocker and Restock_Request. This is a one-to-many relationship because one restocker can receive many restock requests, but each request is sent to exactly one restocker. Its FK is restock_request.restocker_id, referencing restocker(user_id).

BasedOn

- BasedOn is a relationship between Recipe and Restock_Request. This is a one-to-many relationship because one recipe can be the basis for many restock requests, but each request is based on zero or one recipe. Its FK is restock_request.recipe_id, referencing recipe(recipe_id); it's nullable because a simple restock request has no recipe behind it.

Places

- Places is a relationship between Consumer and OGS_Order. This is a one-to-many relationship because one consumer can place many orders, but each order belongs to exactly one consumer. Its FK is ogs_order.consumer_id, referencing consumer(user_id)

Supplies

- Supplies is a relationship between Restocker and Ingredient. This is a many-to-many relationship because one restocker can supply many ingredients, and one ingredient can be supplied by many restockers. Its PK is (restocker_id, ing_id) and its FKs are restocker_id, referencing restocker(user_id), and ing_id, referencing ingredient(ing_id). The table also holds price and qty, which describe that specific restocker-ingredient pairing.

Keeps

- Keeps is a relationship between Consumer and Ingredient. This is a many-to-many relationship because one consumer can keep many ingredients in their pantry, and one ingredient can be kept by many consumers. Its PK is (consumer_id, ing_id) and its FKs are consumer_id, referencing consumer(user_id), and ing_id, referencing ingredient(ing_id).

Stock

- Stock is a relationship between Restaurant and Ingredients. This is a many-to-many relationship because one restaurant can keep many ingredients in their inventory, and one ingredient can be kept by many restaurants. Its PK is (restaurant_id, ing_id) and its FKs are restaurant_id, referencing restaurant(restaurant_id), and ing_id, referencing ingredient(ing_id).

Require

- Require is a relationship between Recipe and Ingredient. This is a many-to-many relationship because one recipe can require many ingredients, and one ingredient can be required by many recipes. Its PK is (recipe_id, ing_id) and its FKs are recipe_id, referencing recipe(recipe_id), and ing_id, referencing ingredient(ing_id).

Contain

- Contain is a relationship between Restock_Request and Ingredient. This is a many-to-many relationship because one restock request can contain many ingredients, and one ingredient can appear in many restock requests. Its PK is (request_id, ing_id) and its FKs are request_id, referencing restock_request(request_id), and ing_id, referencing ingredient(ing_id).

Includes

- Includes is a relationship between OGS_Order and Ingredient. This is a many-to-many relationship because one order can have many ingredients, and one ingredient can be in many orders. Its PK is (order_id, ing_id) and its FKs are order_id, referencing ogs_order(order_id), and ing_id, referencing ingredient(ing_id).

Schemas

Users(user_id, email, password, name, status, created_at)
- PK: user_id

Admin(user_id)
- PK/FK: user_id

Chef(user_id)
- PK/FK: user_id

Restocker(user_id, company_name, approved_by, approved_at)
- PK/FK: user_id

- FK: approved_by

Consumer(user_id)
- PK/FK: user_id

Ingredient(ing_id, name, default_unit, restaurant_qty_on_hand, restaurant_last_updated)
- PK: ing_id

Recipe(recipe_id, name, description, created_at, is_public, created_by)
- PK: recipe_id
- FK: created_by

Recipe_Ingredient(recipe_id, ing_id, qty, unit)
- PK: (recipe_id, ing_id)
- FK: recipe_id, ing_id

Consumer_Inventory(consumer_id, ing_id, qty_avai, unit, last_update)
- PK: (consumer_id, ing_id)
- FK: consumer_id, ing_id

Restocker_Catalog(restocker_id, ing_id, price, qty)
- PK: (restocker_id, ing_id)
- FK: restocker_id, ing_id

Restock_Request(request_id, status, created_at, responded_at, chef_id, restocker_id, recipe_id)
- PK: request_id
- FK: chef_id, restocker_id, recipe_id

Restock_Request_Item(request_id, ing_id, qty_req)
- PK: (request_id, ing_id)
- FK: request_id, ing_id

OGS_Order(order_id, status, created_at, consumer_id)
- PK: order_id
- FK: consumer_id

OGS_Order_Item(order_id, ing_id, qty_ordered)
- PK: (order_id, ing_id)
- FK: order_id, ing_id

Audit_Log(log_id, user_id, entity_type, entity_id, action, created_at)
- PK: log_id
- FK: user_id

Screenshots

users

user_id	email	password	name	status	created_at
1	david@whatsonthene...	password101	David A	ACTIVE	2026-04-23 11:00:00
2	michael@whatsonthe...	password102	Michael B	ACTIVE	2026-05-07 14:50:00
3	andrew@whatsonthen...	password103	Andrew C	ACTIVE	2026-05-07 16:30:00
4	peter@whatsonthene...	password104	Peter D	ACTIVE	2026-05-08 11:30:00
5	james@whatsonthene...	password105	James E	ACTIVE	2026-05-17 16:45:00
6	brian@whatsonthene...	password106	Brian F	ACTIVE	2026-05-23 15:30:00
7	liam@whatsonthenem...	password107	Liam G	ACTIVE	2026-05-26 12:30:00
8	oliver@whatsonthene...	password108	Oliver H	ACTIVE	2026-05-30 08:30:00
9	will@whatsonthenem...	password109	Will I	ACTIVE	2026-06-02 06:10:00
10	mary@whatsonthene...	password110	Mary K	ACTIVE	2026-06-12 19:20:00
11	charles@whatsonthen...	password111	Charles K	ACTIVE	2026-06-15 10:00:00
12	daniel@whatsonthene...	password112	Daniel L	ACTIVE	2026-06-16 11:30:00
13	edward@whatsonthen...	password113	Edward M	ACTIVE	2026-06-17 13:00:00

admin

user_id
1
2
3
4
5
6
7
8
9
10
NULL

chef

user_id	restaurant_id
11	1
12	2
13	3
14	4
15	5
16	6
17	7
18	8
19	9
20	10
NULL	NULL

Restaurant

restaurant_id	name
1	The Golden Spoon
2	Fisher Grill
3	Boiling Lobster
4	BBQ Restaurant
5	The Table
6	Blue Kitchen
7	Tree Restaurant
8	Seafood Dinner
9	HotPot
10	Pizza Place
NULL	NULL

restocker

user_id	company_name	approved_by	approved_at
21	Bay Fresh Produce	1	2026-08-02 18:30:00
22	Golden State Dairy	2	2026-07-20 23:30:00
23	Pacific Pantry Wholesale	3	2026-08-15 12:40:00
24	Sunrise Farms Co-op	1	2026-07-25 15:55:00
25	Silicon Valley Meats	2	2026-08-09 21:20:00
26	Harvest Moon Grains	3	2026-07-31 19:05:00
27	Mission Spice Traders	1	2026-07-22 10:50:00
28	Coastal Foods Supply	2	2026-08-02 22:45:00
29	Redwood Organics	3	2026-08-01 13:15:00
30	NorCal Provisions	NULL	NULL
NULL	NULL	NULL	NULL

consumer

user_id
31
32
33
34
35
36
37
38
39
40
NULL

ingredients

ing_id	name	default_unit	created_at
1	all-purpose flour	g	2026-09-01 08:00:00
2	sugar	g	2026-09-01 11:00:00
3	eggs	count	2026-09-03 09:15:00
4	butter	g	2026-09-04 10:00:00
5	whole milk	ml	2026-09-05 11:30:00
6	olive oil	ml	2026-09-06 18:00:00
7	salt	g	2026-09-07 13:00:00
8	black pepper	g	2026-09-08 14:20:00
9	garlic cloves	count	2026-09-09 08:45:00
10	yellow onion	count	2026-09-10 15:10:00
11	tomato	count	2026-09-11 09:30:00
12	chicken breast	g	2026-09-12 07:30:00
13	ground beef	g	2026-09-13 12:00:00
14	spaghetti	g	2026-09-14 10:15:00

recipes

recipe_id	name	description	created_at	is_public	crea
1	Spaghetti Carbonara	Classic Roman pasta ...	2026-06-01 08:00:00	1	11
2	Margherita Pizza	Thin-crust pizza with t...	2026-06-05 09:20:00	1	12
3	Vegetable Fried Rice	Quick wok-fried rice ...	2026-06-10 11:10:00	1	13
4	Creamy Mushroom Pa...	Spaghetti in a garlic ...	2026-06-15 14:30:00	1	14
5	Classic Pancakes	Fluffy breakfast panca...	2026-06-20 08:45:00	1	15
6	Lemon Garlic Chicken	Pan-seared chicken br...	2026-06-25 19:10:00	1	16
7	Bellini House Carbonara	Chef Marco's larger-p...	2026-07-05 12:20:00	0	11
8	House Beef Tacos	Staff-meal tacos.	2026-07-15 10:40:00	0	12
9	Garlic Mashed Potatoes	Creamy mashed potat...	2026-07-25 20:15:00	0	13
10	Veggie Rice	Rice with vegetables	2026-08-01 18:30:00	0	14
11	Mushroom Pasta	Pasta served with mu...	2026-08-10 07:50:00	0	15
12	Sunday Pancakes	Pancake for weekends	2026-08-20 16:10:00	0	16
13	Quick Lemon Chicken	Quickly cooked chicken wi	2026-09-01 13:25:00	0	17

recipe_ingredients

recipe_id	ing_id	qty	unit
1	3	2.00	count
1	7	3.00	g
1	8	2.00	g
1	14	200.00	g
1	15	50.00	g
1	26	100.00	g
2	1	300.00	g
2	6	30.00	ml
2	7	5.00	g
2	11	3.00	count
2	16	200.00	g
2	17	10.00	g
3	3	2.00	count

consumer_inventory

consumer_id	ing_id	qty_avai	unit	last_update
31	3	12.00	count	2026-09-21 12:10:00
31	9	6.00	count	2026-09-21 12:10:00
31	10	3.00	count	2026-09-20 10:05:00
31	18	1000.00	g	2026-09-27 14:35:00
31	19	250.00	ml	2026-09-26 19:00:00
31	23	4.00	count	2026-09-18 17:45:00
32	4	200.00	g	2026-09-24 17:10:00
32	7	300.00	g	2026-09-15 15:15:00
32	9	6.00	count	2026-09-24 11:15:00
32	14	400.00	g	2026-09-17 14:30:00
32	15	100.00	g	2026-09-19 17:00:00
32	25	100.00	ml	2026-09-20 11:40:00
32	27	150.00	g	2026-09-23 09:30:00

Restaurant_inventory

restaurant_id	ing_id	qty_on_hand	unit	last_updated
1	3	24.00	count	2026-10-11 13:47:55
1	4	20.00	tbsp	2026-10-11 13:47:55
1	12	5.00	count	2026-10-11 13:47:55
1	14	2000.00	g	2026-10-11 13:47:55
1	24	12.00	count	2026-10-11 13:47:55
2	1	1500.00	g	2026-10-11 13:50:56
2	7	2000.00	g	2026-10-11 13:50:56
3	5	3000.00	ml	2026-10-11 13:50:56
4	9	15.00	count	2026-10-11 13:50:56
5	13	1000.00	g	2026-10-11 13:50:56
NULL	NULL	NULL	NULL	NULL

restocker_catalog

restocker_id	ing_id	price	qty
21	1	0.0023	15000.00
21	10	0.0950	45.00
21	19	0.0065	8000.00
21	28	0.0090	6000.00
22	2	0.0021	9000.00
22	11	0.5800	32.00
22	20	0.0120	7000.00
22	29	0.1300	5500.00
23	3	0.3400	55.00
23	12	0.0118	12000.00
23	21	0.9500	38.00
23	30	0.0150	14000.00
24	4	0.0110	11000.00

restock_request

request_id	status	created_at	responded_at	chef_id	restocker_id	reci
1	FULFILLED	2026-09-10 07:00:00	2026-09-10 11:15:00	11	23	7
2	ACCEPTED	2026-09-18 20:05:00	2026-09-19 00:20:00	11	27	8
3	PENDING	2026-09-28 09:50:00	NULL	13	26	9
4	FULFILLED	2026-09-05 14:50:00	2026-09-05 19:05:00	12	28	NULL
5	REJECTED	2026-09-20 11:15:00	2026-09-20 14:30:00	14	25	NULL
6	ACCEPTED	2026-09-24 18:05:00	2026-09-24 21:20:00	15	23	NULL
7	PENDING	2026-09-29 19:10:00	NULL	16	21	NULL
8	REJECTED	2026-09-15 20:00:00	2026-09-15 21:15:00	17	21	NULL
9	FULFILLED	2026-09-12 15:05:00	2026-09-12 20:20:00	18	23	NULL
10	PENDING	2026-09-27 09:20:00	NULL	11	24	NULL
11	PENDING	2026-09-29 20:45:00	NULL	12	22	NULL
12	FULFILLED	2026-09-08 15:00:00	2026-09-08 18:15:00	13	21	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL

restock_request_item

request_id	ing_id	qty_req
1	7	3.00
1	8	2.00
1	26	100.00
2	10	1.00
2	11	2.00
2	30	80.00
3	4	40.00
3	5	100.00
3	22	4.00
4	8	2000.00
4	17	500.00
5	5	5000.00
5	6	500.00

ogs_order

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

order_id	status	created_at	consumer_id
1	FULFILLED	2026-09-01 08:30:00	31
2	PENDING	2026-09-03 10:15:00	31
3	FULFILLED	2026-09-05 12:40:00	32
4	CANCELLED	2026-09-07 14:05:00	33
5	FULFILLED	2026-09-09 16:20:00	34
6	PENDING	2026-09-11 09:45:00	35
7	FULFILLED	2026-09-14 15:15:00	36
8	FULFILLED	2026-09-17 11:50:00	37
9	PENDING	2026-09-20 07:05:00	38
10	FULFILLED	2026-09-23 14:15:00	39
11	CANCELLED	2026-09-26 15:50:00	40
12	PENDING	2026-09-29 10:20:00	32
NULL	NULL	NULL	NULL

ogs_order_item

order_id	ing_id	qty_ordered
1	5	1000.00
1	9	36.00
1	12	500.00
2	3	24.00
2	5	500.00
2	17	1000.00
3	25	500.00
3	27	2000.00
4	15	500.00
4	20	100.00
5	1	400.00
5	12	200.00
5	16	400.00

audit_log

log_id	user_id	entity_type	entity_id	action	created_at
1	1	RESTOCKER	24	APPROVE_RESTOCKER	2026-07-20 09:00:00
2	1	RESTOCKER	27	APPROVE_RESTOCKER	2026-07-22 10:50:00
3	3	RESTOCKER	26	APPROVE_RESTOCKER	2026-07-25 15:55:00
4	3	RESTOCKER	29	APPROVE_RESTOCKER	2026-07-31 19:05:00
5	2	RESTOCKER	22	APPROVE_RESTOCKER	2026-08-01 13:15:00
6	1	RESTOCKER	21	APPROVE_RESTOCKER	2026-08-02 18:30:00
7	2	RESTOCKER	28	APPROVE_RESTOCKER	2026-08-02 22:45:00
8	2	RESTOCKER	25	APPROVE_RESTOCKER	2026-08-09 21:20:00
9	3	RESTOCKER	23	APPROVE_RESTOCKER	2026-08-15 12:40:00
10	2	INGREDIENT	2	CATALOG_EDIT	2026-09-03 16:20:00
11	12	RESTOCK_...	4	SUBMIT_REQUEST	2026-09-05 14:50:00
12	28	RESTOCK_...	4	ACCEPT_REQUEST	2026-09-05 19:05:00
13	13	RESTOCK_...	12	SUBMIT_REQUEST	2026-09-08 15:00:00